

SSO + Managed Identity + AKS

Architecture notes for the sso-aks-demo learning app

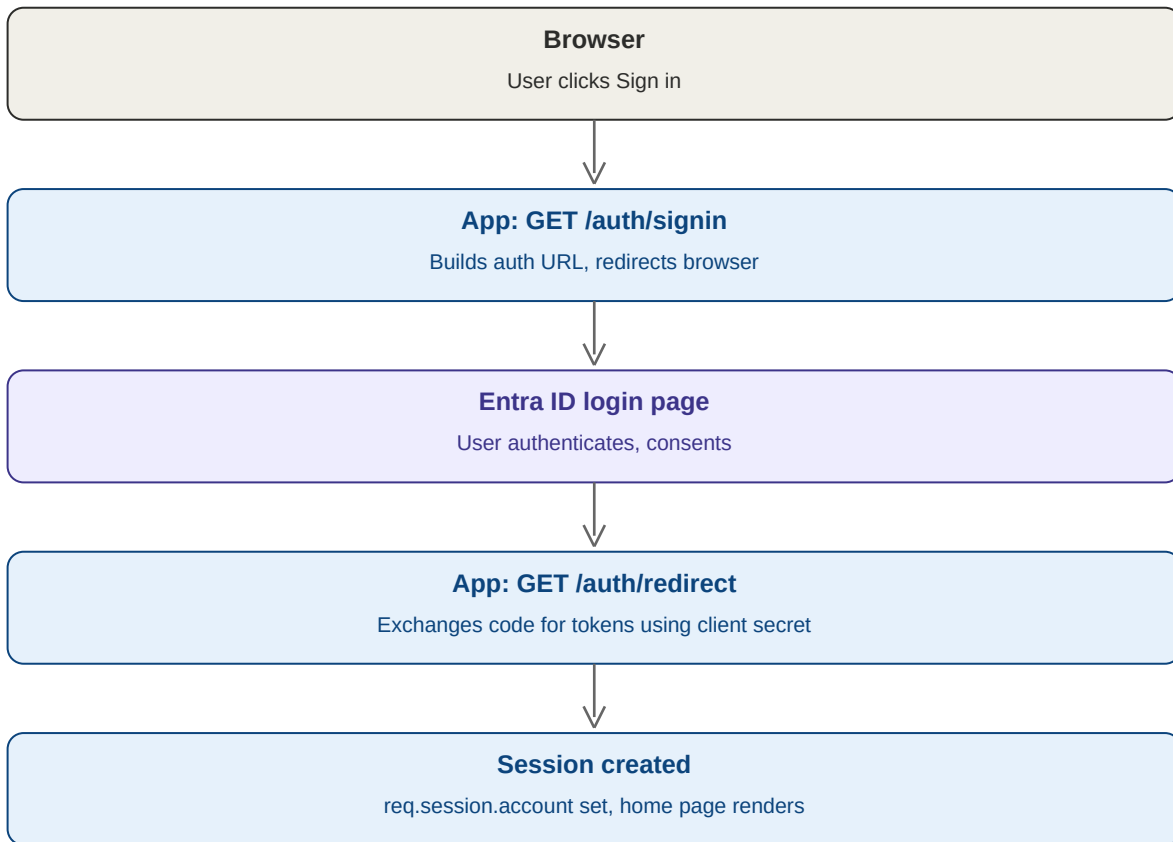
Overview

This app demonstrates two independent trust chains that get wired together: **SSO** (a user proving who they are to the app, via Microsoft Entra ID) and **Managed Identity** (the app proving what it itself is allowed to do, via Azure's workload identity system). The two never mix — a signed-in user's session cannot be used to reach Key Vault, and the app's Key Vault permissions cannot be used to impersonate a user. That separation is the core idea behind the whole pattern.

In local development, the app's own identity is your `az login` session. Once deployed to AKS, that identity is replaced by a User-Assigned Managed Identity, federated to a Kubernetes ServiceAccount via Azure Workload Identity — with no code change required.

Part 1 — SSO authentication flow

This is what happens when a user clicks “Sign in”. It is entirely about proving the identity of the *human* using the browser.



Step-by-step

1. Sign-in request — GET /auth/signin builds a URL to login.microsoftonline.com using MSAL's `getAuthCodeUrl()`, embedding the app's client ID, requested scopes, and the exact redirect URI registered in Entra ID. The server then issues a 302 redirect — the browser leaves the app entirely at this point.

2. Microsoft handles login — Entra ID shows its own login page, handles password and MFA, and checks the redirect URI against what was registered on the app. None of this touches the app's server.

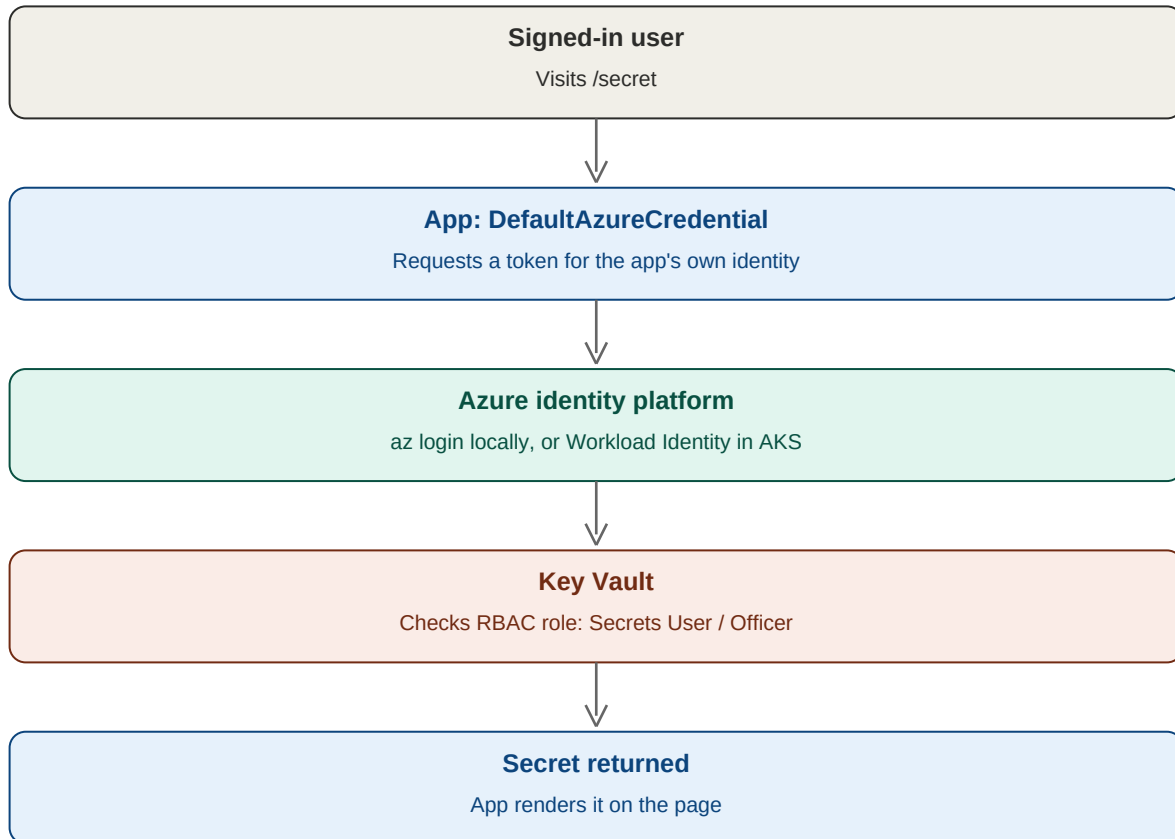
3. Redirect back with a code — On success, Microsoft redirects the browser to /auth/redirect with a short-lived, single-use authorization code in the query string. This code alone cannot be used to access anything — it must be redeemed by the app.

4. Server-side token exchange — The app calls MSAL's `acquireTokenByCode()`, which makes a direct server-to-server call to Microsoft's token endpoint, sending the code plus the app's client ID and client secret. This exchange can only happen server-side because the secret must never reach the browser.

5. Session established — The app extracts the user's name, username, and ID token claims from the response and stores them in `req.session.account` (via `express-session`). The browser only ever holds a small signed cookie referencing that session — not the tokens themselves.

Part 2 — Managed Identity to Key Vault

This happens when a signed-in user visits `/secret`. It is entirely about proving what the *app itself* (the workload) is allowed to do — the user's identity plays no role here at all.



Step-by-step

1. Request arrives — `requireAuth` middleware confirms `req.session.account` exists (from Part 1); otherwise the user is redirected back to sign in.

2. DefaultAzureCredential — `keyvault.js` constructs a `SecretClient` using `DefaultAzureCredential`, which tries several credential sources in order: environment variables, Workload Identity (a federated token, in AKS), Managed Identity, then Azure CLI login — whichever is available in the current environment.

3. Token acquisition — Locally, this resolves to your own `az login` session. In AKS, the Azure Workload Identity webhook has already injected a federated token into the pod, tied to its `ServiceAccount`, with no secret ever stored in the cluster.

4. RBAC check — Key Vault validates the caller's role assignment before allowing the operation — Key Vault Secrets User (read) or Key Vault Secrets Officer (read/write). No role assignment means an immediate Forbidden response, regardless of who the signed-in application user is.

5. Secret returned — On success, the secret value is returned to the app and rendered on the `/secret` page. No connection string or API key was ever stored in app code, environment files, or Kubernetes secrets to make this call.

Concept recap

Concept	Where it lives in the app
SSO (OIDC Authorization Code flow)	src/authConfig.js, src/routes/auth.js
Managed Identity	src/keyvault.js (DefaultAzureCredential)
Workload Identity federation	k8s/01-serviceaccount.yaml, federated credential on the User-Assigned Managed Identity
AKS deployment	Dockerfile, k8s/04-deployment.yaml, k8s/05-service.yaml, k8s/06-ingress.yaml

Production hardening notes

- Use a certificate instead of a client secret for the confidential client.
- Validate the OAuth state parameter on the redirect callback to prevent CSRF.
- Store the client secret in Key Vault and sync it into the cluster with the Secrets Store CSI Driver, rather than `kubectl create secret`.
- Use a shared session store (e.g. Redis) instead of in-memory sessions once running multiple replicas.
- Set `cookie.secure = true` and `app.set('trust proxy', 1)` once behind TLS-terminating ingress.